Validation

Chapter 12

Randy Connolly and Ricardo Hoar

© 2015 Pearson

Validazione

La validazione dei dati inseriti dall'utente deve essere fatta sia lato client che lato server.

Checking user input

Notice that this parameter has no value.

Example query string: `id=0&name1=&name2=smith&name3=%20`

This parameter's value is a space character (URL encoded).

`isset($_GET['id'])`	returns	**true**
`isset($_GET['name1'])`	returns	**true**
`isset($_GET['name2'])`	returns	**true**
`isset($_GET['name3'])`	returns	**true**
`isset($_GET['name4'])`	returns	**false**

Notice that a missing value for a parameter is still considered to be `isset`.

Notice that only a missing parameter name is considered to be not `isset`.

`empty($_GET['id'])`	returns	**true**
`empty($_GET['name1'])`	returns	**true**
`empty($_GET['name2'])`	returns	**false**
`empty($_GET['name3'])`	returns	**false**
`empty($_GET['name4'])`	returns	**true**

Notice that a value of zero is considered to be empty. This may be an issue if zero is a "legitimate" value in the application.

Notice that a value of space is considered to be **not** empty.

Checking user input

Checking for a number

```
$id = $_GET['id'];
if (!empty($id) && is_numeric($id) ) {
   // use the query string since it exists and is a numeric value
   ...
}
```

LISTING 12.1 Testing a query string to see if it exists and is numeric

Section 4 of 6

REGULAR EXPRESSIONS

Regular Expressions

A **regular expression** is a set of special characters that define a pattern.

They are a type of language that is intended for the matching and manipulation of text.

Regular expressions are a concise way to eliminate the conditional logic that would be necessary to ensure that input data follows a specific format.

Le espressioni regolari

- Utili per:
  - verificare se un testo rispetta un certo formato;
  - estrarre parti di testo;
  - sostituire porzioni di stringa.
- Nel Web sono particolarmente utili per validare l'input dell'utente
- Disponibili in praticamente tutti i linguaggi di programmazione (JavaScript, PHP, Java, C++, ...), numerosi tool (grep, editor di testi) e anche applicazioni (MySQL, PostgreSQL, ...)

Regular Expressions

Syntax

A regular expression consists of two types of characters: **literals** and **metacharacters**.

- A **literal** is a character you wish to match in the target
- A **metacharacter** is a special symbol that acts as a command to the regular expression parser

Intro

- Nella sua forma più semplice un'espressione regolare è una sequenza di caratteri che matcha sé stessa

  abc → dsa d d d asd abc vvbn abckabc vvv

- Oltra ai caratteri normali, quelli che rappresentano sé stessi, ce ne sono altri speciali che rappresentano gruppi di caratteri, posizioni, quantità, ...

Regular Expressions

Characters with Special Meaning

.	[	]	\	(	)	^	$	|	*	?	{	}	+

TABLE 12.2 Regular Expression Metacharacters (i.e., Characters with Special Meaning)

To use a metacharacter as a literal, you will need to escape it by prefacing it with a backslash (\)

Sintassi di base

I caratteri "normali" rappresentano sé stessi.
Altri caratteri rappresentano gruppi di caratteri, posizioni, o quantità.

Simbolo	Significato	Esempio	Match
`.`	Qualsiasi carattere singolo (tranne \n)	`a.c`	`abc`, `axc`
`*`	Ripetizione 0 o più volte	`go*gle`	`ggle`, `google`
`+`	Ripetizione 1 o più volte	`ha+`	`ha`, `haa`, `haaa`
`?`	0 o 1 volta	`colou?r`	`color`, `colour`
`^`	Inizio stringa	`^Ciao`	`Ciao mondo`
`$`	Fine stringa	`fine$`	`Ecco la fine`

Classi di caratteri

- Le classi individuano i gruppi di caratteri possibili.
- Alcuni simboli rappresentano gruppi di caratteri

Classe	Significato	Esempio
`[abc]`	a, b o c	`c[aeiou]sa` → casa, cesa
`[^abc]`	tutto tranne a, b, c	`[^0-9]` → nessuna cifra
`[a-z]`	tutte le lettere minuscole	
`[A-Z0-9]`	lettere maiuscole o numeri	

Simbolo	Significato	Esempio
`\d`	cifra, equivale a [0–9]	`\d{3}` → 3 cifre
`\w`	lettera o cifra, equivale [a-zA-Z0-9_]	`\w+` → sequenza di lettere, cifre, undersc. (almeno una)
`\s`	spazio o tab o rit. carr. o nuova linea	
`\D`, `\W`, `\S`	negazioni	opposti dei precedenti

Quantificatori

- Determinano il numero di occorrenze

Notazione	Significato	Esempio
`{n}`	Esattamente n	`\d{4}` → 4 cifre
`{n,}`	Almeno n	`a{2,}` → `aa`, `aaa`
`{n,m}`	Da n a m	`\d{2,5}` → da 2 a 5 cifre
?	0 o 1	abc? -> ab, abc
*	0 o più	abc* -> ab, abc, abcc, abccc, ...
+	1 o più	abc+ -> abc, abcc, abccc, ...

Nov(ember)? matcha Nov and November
Feb(ruary)? 23(rd)? matcha February 23rd, February 23, Feb 23rd e Feb 23.
<[A-Za-z][A-Za-z0-9]*> matcha i tag html senza attributi.

Quantificatori

- I quantificatori cercano di catturare più testo possibile (comportamento greedy).
- Supponiamo di usare <.+> per trovare tag html nella stringa

Questa è una <EM>prova</EM> sulle regex

viene trovata *<EM>prova</EM>* e non *<EM>* e *</EM>* come magari vorremmo

- Il + è greedy: dopo il primo < matcha tutti i caratteri della stringa fino ad arrivare in fondo, quindi fallisce. Si torna quindi indietro per far matchare cosa segue il + nella regex (il >)
- Il comportamento può essere cambiato da greedy a lazy con ?

<.+?>

dopo il match di < cerca un carattere qualunque lo trova, poi cerca > fallisce, torna indietro, cerca due caratteri qualunque, li trova, cerca > lo trova...

Ancore e posizioni

- Le ancore non corrispondono a caratteri, ma a **posizioni** nella stringa.

Simbolo	Significato	Esempio
`^`	inizio stringa	`^Ciao`
`$`	fine stringa	`fine$`
`\b`	confine di parola	`\bcat\b` → "cat" ma non "concatenate"
`\B`	non confine	inverso di `\b`

^a to abc matches a
^b does not match abc at all
c$ matches c in abc, while a$ does not match at all
\b4\b non ha match in 44 sheets of a4

Gruppi

- (abc) → gruppo catturante
- \1, \2, … → riferimento a gruppi precedenti

(\d{2})/(\d{2})/(\d{4})

15/11/2025
Gruppi catturati:

15 (giorno)

11 (mese)

2025 (anno)

Possono essere riferiti con \1, \2, \3

([a-c]**)**x**\1**x**\1** matches axaxa, bxbxb and cxcxc

Gruppi

- Lookhaed
- q(?=u) matches a q that is followed by a u, without making the u part of the match.

Regular Expressions

Table of typical patterns

Expression	Description
^ … $	If used at the very start and end of the regular expression, it means that the entire string (and not just a substring) must match the rest of the regular expression contained between the ^ and the $ symbols.
\t	Matches a tab character.
\n	Matches a new line character.
.	Matches any character other than \n.
[qwerty]	Matches any single character of the set contained within the brackets.
[^qwerty]	Matches any single character not contained within the brackets.
[a-z]	Matches any single character within range of characters.
\w	Matches any word character. Equivalent to [a-zA-Z0-9].
\W	Matches any nonword character.
\s	Matches any white-space character.
\S	Matches any nonwhite-space character.
\d	Matches any digit.
\D	Matches any nondigit.
*	Indicates zero or more matches.
+	Indicates one or more matches.
?	Indicates zero or one match.
{n}	Indicates exactly n matches.
{n,}	Indicates n or more matches.
{n,m}	Indicates at least n but no more than m matches.
|	Matches any one of the terms separated by the | character. Equivalent to Boolean OR.
()	Groups a subexpression. Grouping can make a regular expression easier to understand.

Regular Expressions

Building one example

Consider a regular expression that would match a North American phone number without the area code.

A valid number contains three numbers, followed by a dash, followed by four numbers without any other character.

The regular expression for this would be:

```
^\d{3}–\d{4}$
```

Regular Expressions

three numbers, followed by a dash, followed by four numbers

^\d{3}–\d{4}$

- The dash is here a literal character (can be used as a metacharacter in ranges but not here as it is not within **[]**); the rest are all metacharacters
- The **^** and **$** symbol indicate the beginning and end of the string, respectively
- The metacharacter **\d** indicates a digit, while the metacharacters **{3}** and **{4}** indicate three and four repetitions of the previous match (i.e., a digit), respectively

Regular Expressions

three numbers, followed by a dash, followed by four numbers

A more sophisticated regular expression for a phone number would not allow the first digit in the phone number to be a zero ("0") or a one ("1").

The modified regular expression for this would be:

```
^[2-9]\d{2}–\d{4}$
```

Regular Expressions

Any number (but 0,1), then 2 more, a dash and 4 more.

^[2-9]\d{2}–\d{4}$

- The **[2-9]** metacharacter indicates that the first character must be a digit within the range 2 through 9
- Since only two more numbers are needed the pattern **\d{3}** becomes **\d{2}**

Regular Expressions

Allow a space, period, or dash in the number.

We can make our regular expression a bit more flexible by allowing either a single space (440 6061), a period (440.6061), or a dash (440-6061) between the two sets of numbers.

We can do this via the **[]** metacharacter:

```
^[2-9]\d{2}[–\s\.]\d{4}$
```

Regular Expressions

Allow a space, period, or dash in the number.

^[2-9]\d{2}[–\s\.]\d{4}$

This expression indicates that the fourth character in the input must match one of the three characters contained within the square brackets

– matches a dash (since it is at the beginning of the character class [] there is no need to escape it), **\s** matches a white space, and **\.** matches a period

^[2-9]\d{2}[\–\s\.]\d{4}$

Same as above. In other cases, we must escape both **-** and **.**

Regular Expressions

Allow multiple spaces

If we want to allow multiple spaces (but only a single dash or period) in our number:

```
^[2-9]\d{2}[–\s\.]\s*\d{4}$
```

The metacharacter sequence **\s*** matches zero or more white spaces.

Regular Expressions

How about area code

To allow the area code to be

- Surrounded by Brackets (403) 440-6061
- Separated with spaces 403 440 6061
- A Dash 403-440-6061
- A Period 403.440.6061

```
^\(?\s*\d{3}\s*[\)–\.]?\s*[2-9]\d{2}\s*[–\.]\s*\d{4}$
```

Regular Expressions

How about area code

^\(?\s*\d{3}\s*[\)–\.]?\s*[2-9]\d{2}\s*[–\.]\s*\d{4}$

The expression now matches

- zero or one "(" characters **\(?**
- zero or more spaces **\s***three digits **\d{3}**
- zero or more spaces **\s***
- either a ")" a "-", or a "." character **[\)-\.]?**
- zero or more spaces **\s***

Regular Expressions

How about area code

Finally, to make the area code optional we will group the area code by surrounding the area code subexpression within grouping metacharacters— which are "(" and ")"— and then make the group optional using the ? metacharacter.

```
^(\(?\s*\d{3}\s*[\)–\.]?\s*)?[2-9]\d{2}\s*[–\.]\s*\d{4}$
```

This may seem frightening, but compare to :

Regular Expressions Alternative

^(\(?\s*\d{3}\s*[\)–\.]?\s*)?[2-9]\d{2}\s*[–\.]\s*\d{4}$

```
var phone=document.getElementById("phone").value;
var parts = phone.split(".");                    // split on .
if (parts.length !=3){
   parts = phone.split("-");                     // split on -
}
if (parts.length == 3) {
   var valid=true;                     // use a flag to track validity
   for (var i=0; i < parts.length; i++) {
      // check that each component is a number
      if (!isNumeric(parts[i])) {
         alert( "you have a non-numeric component");
         valid=false;
      } else { // depending on which component make sure it's in range
         if (i<2) {
            if (parts[i]<100 || parts[i]>999) {
               valid=false;
            }
         else {
            if (parts[i]<1000 || parts[i]>9999) {
               valid=false;
            }
         }
      }
      } // end if isNumeric
      } // end for loop
if (valid) {
   alert(phone + "is a valid phone number");
}
}
alert ("not a valid phone number");
```

LISTING 12.7 A phone number validation script without regular expressions

Esempi

Email	`^[\w.-]+@[\w.-]+\.\w{2,}$`
Numero di telefono	`^\+?\d{9,13}$`
CAP italiano	`^\d{5}$`
Password complessa	`^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$`

Regex is everywhere

Including MySQL

MySQL also supports regular expressions through the REGEXP operator.

For instance, the following SQL statement matches all art works whose title contains one or more numeric digits:

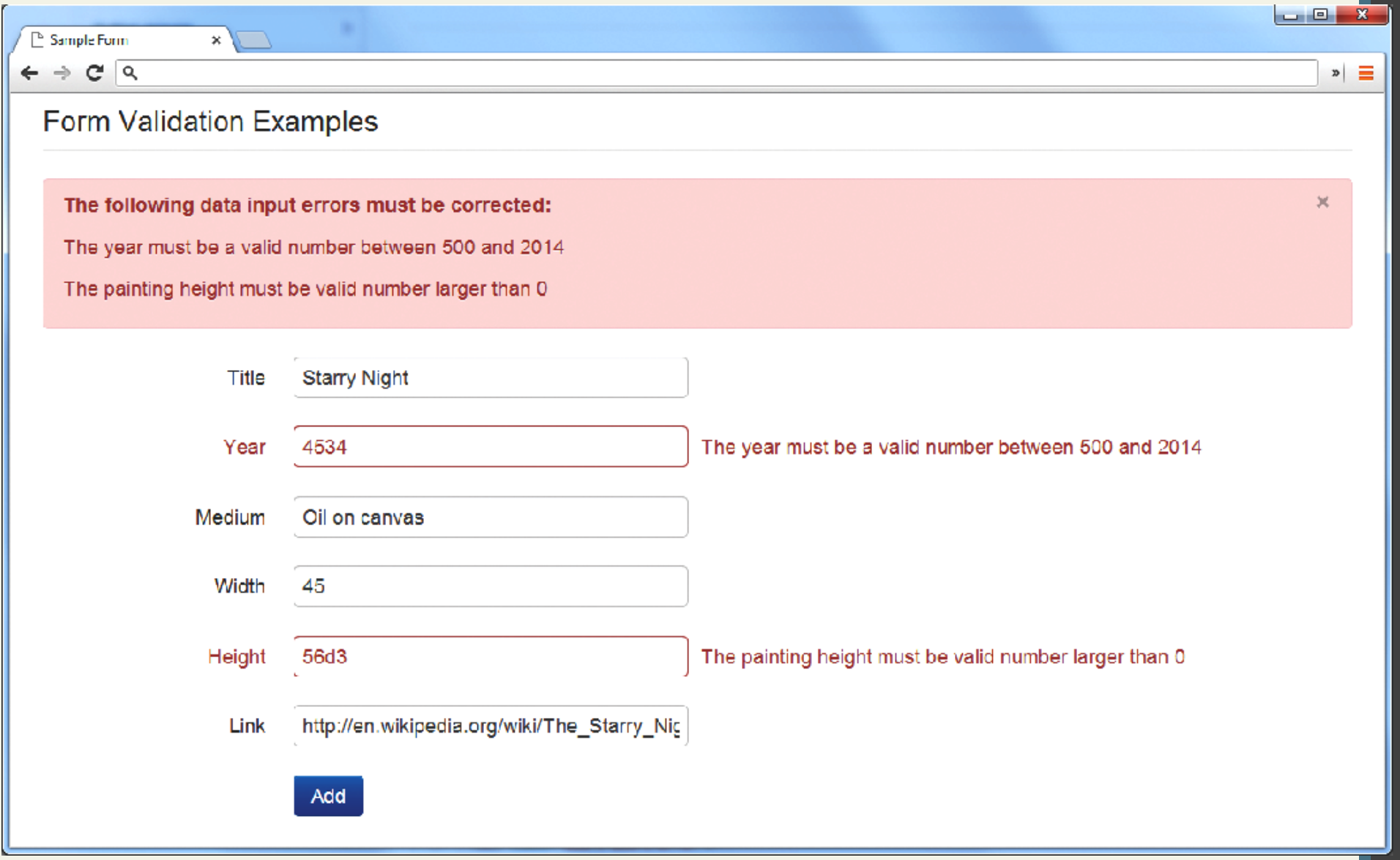
```
SELECT * FROM ArtWorks WHERE Title REGEXP '[0-9]+'
```

Section 5 of 6

VALIDATING USER INPUT

Notifying the User

What's wrong, where is it, and how to fix it.



Types of Input Validation

- **Required information**. Some data fields just cannot be left empty.
- **Correct format**. Some information, such as postal codes, credit card numbers, and social security numbers have to follow certain pattern rules.

Types of Input Validation

Continued

- **Comparison**. Perhaps the most common example of this type of validation is entering passwords: most sites require the user to enter the password twice to ensure the two entered values are identical.
- **Range check**. Information such as numbers and dates have infinite possible values. However, most systems need numbers and dates to fall within realistic ranges.
- **Custom**. Some validations are more complex and are unique to a particular application

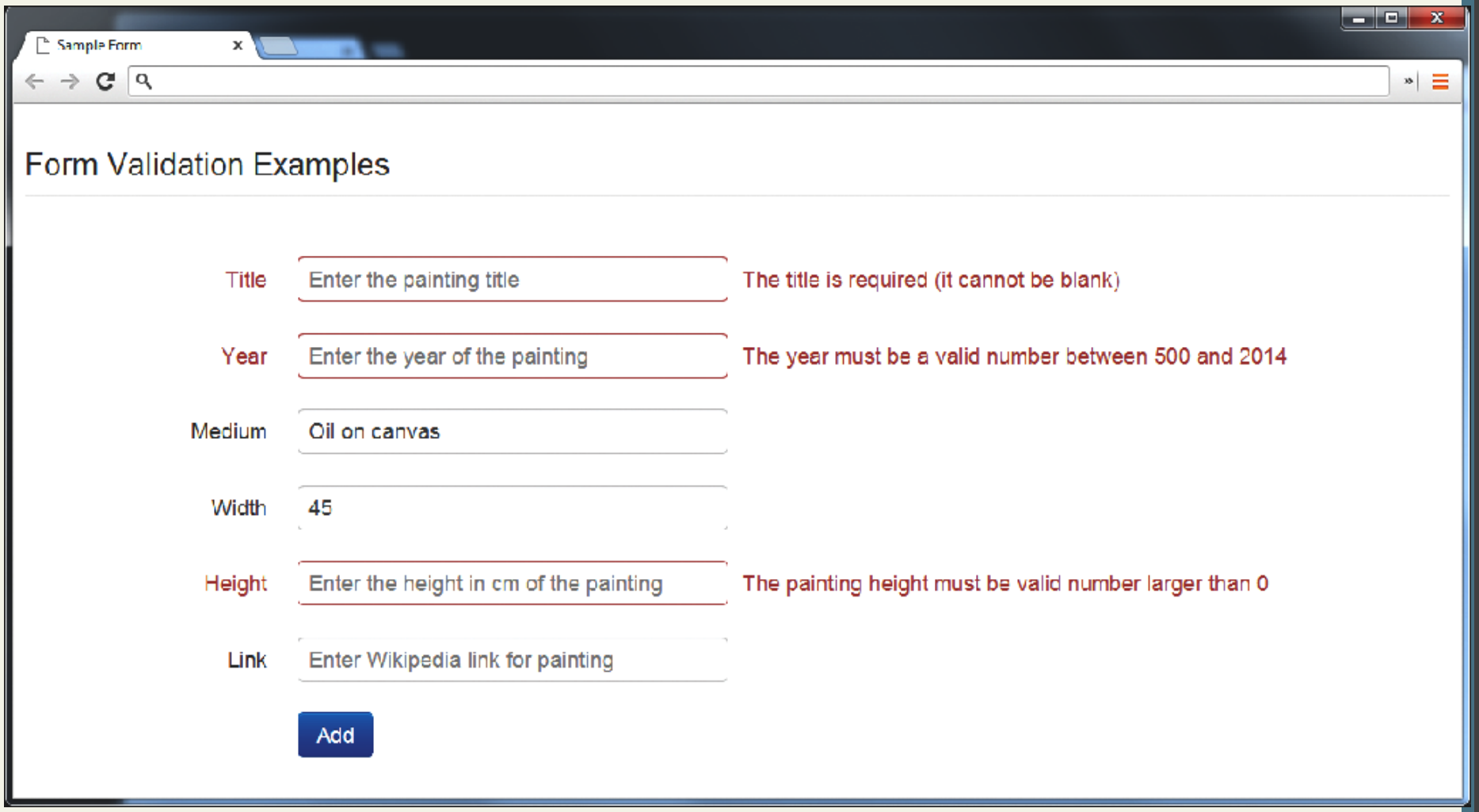
Notifying the User

We found an error, now what?

- **What is the problem?** Users do not want to read lengthy messages to determine what needs to be changed. They need to receive a visually clear and textually concise message.
- **Where is the problem?** Some type of error indication should be located near the field that generated the problem.
- **If appropriate, how do I fix it?** For instance, don't just tell the user that a date is in the wrong format, tell him or her what format you are expecting, such as "The date should be in *yy/mm/dd* format."

Another illustrative examples

What's wrong, where is it, and how to fix it.



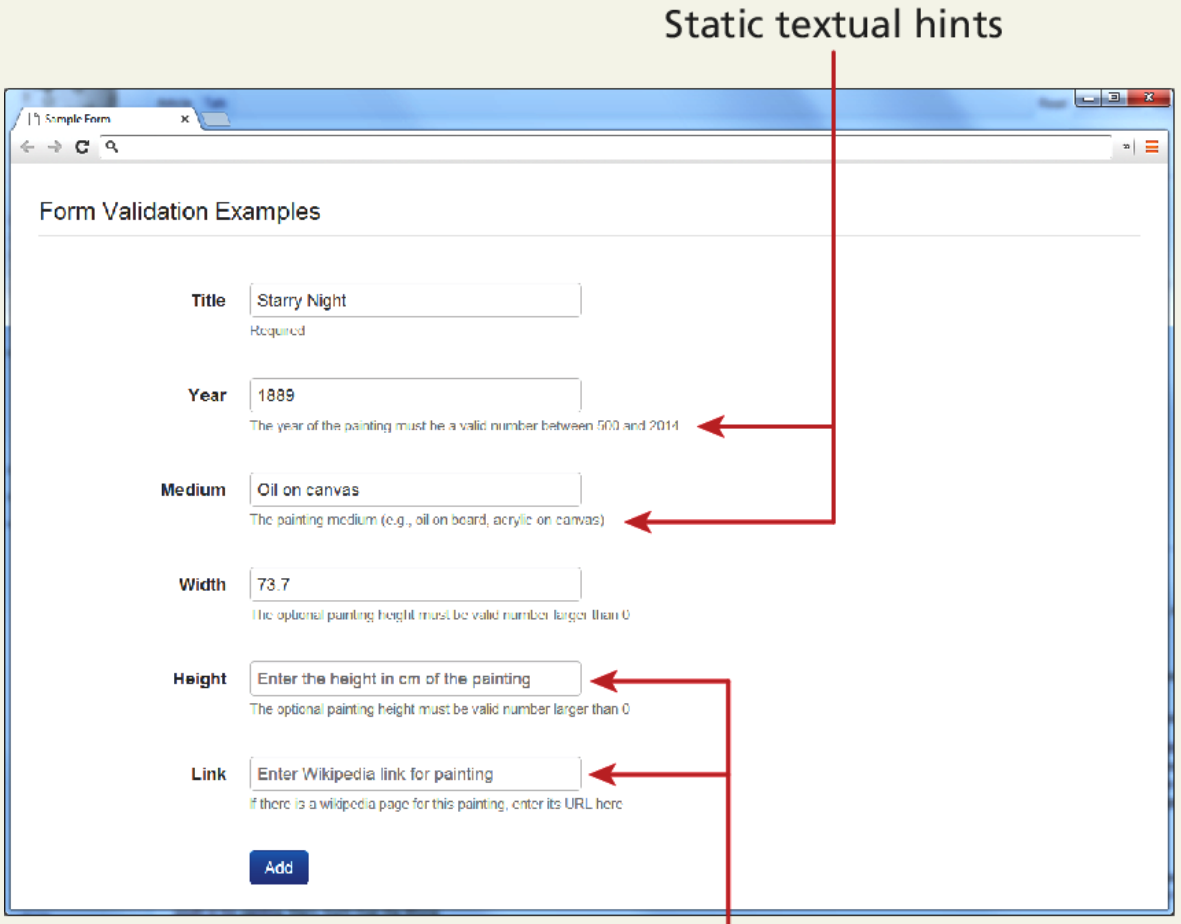
How to reduce validation errors

An ounce of prevention is worth a pound of cure

- Using pop-up JavaScript alert (or other popup) messages
- Provide textual hints to the user on the form itself
- Using tool tips to display context-sensitive help about the expected input

How to reduce validation errors

An ounce of prevention is worth a pound of cure




```
<input type="text" … placeholder="Enter the height ...">
```

How to reduce validation errors

HTML 5 input types

Many user input errors can be eliminated by choosing a better data entry type than the standard

```
<input type="text">
```

If you need a number, use the HTML5

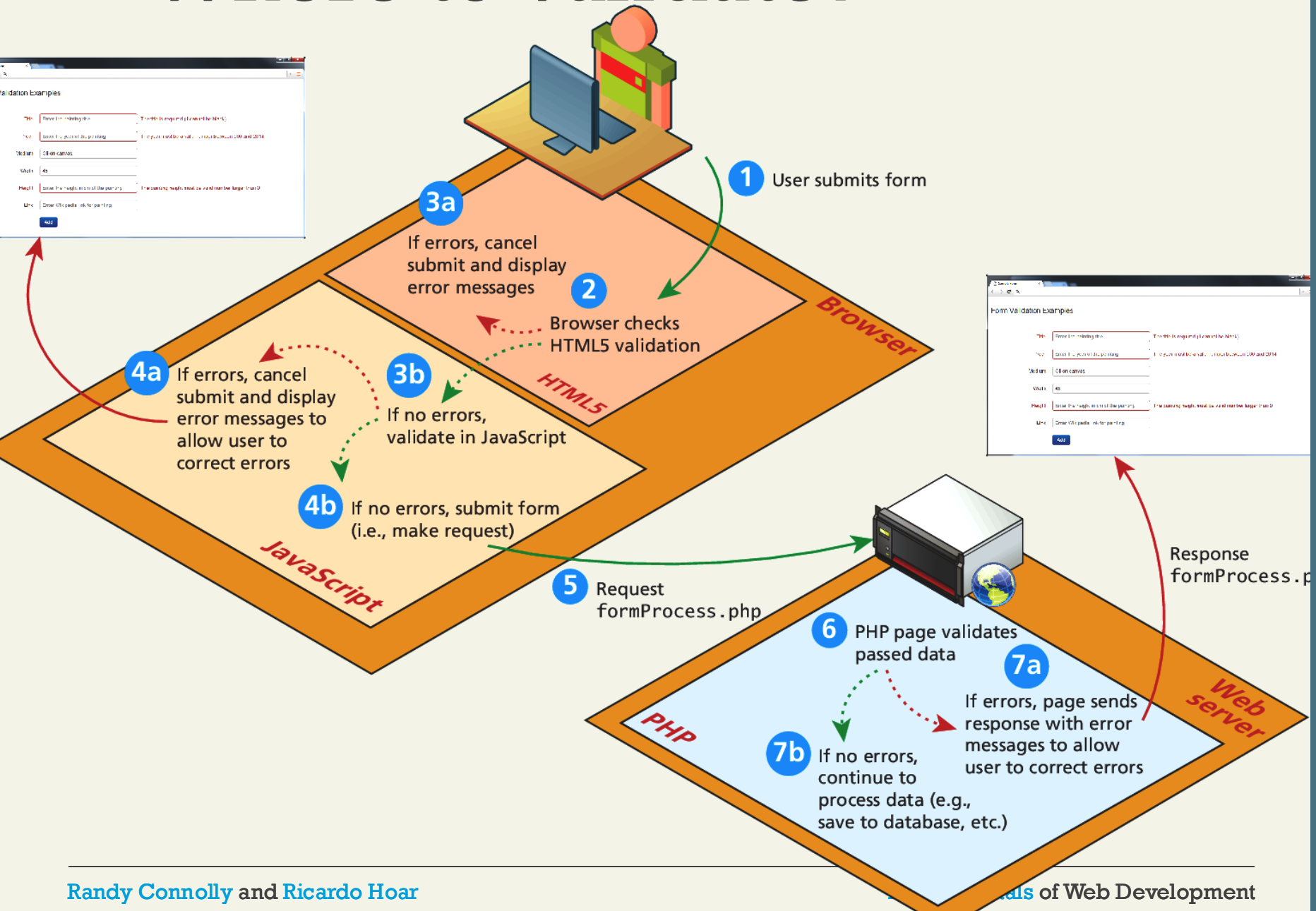
```
<input type="number">
```

Same applies for the other specific input types available in HTML5

Section 6 of 6

WHERE TO PERFORM VALIDATION

Where to Validate?



Where to Validate?

So many places

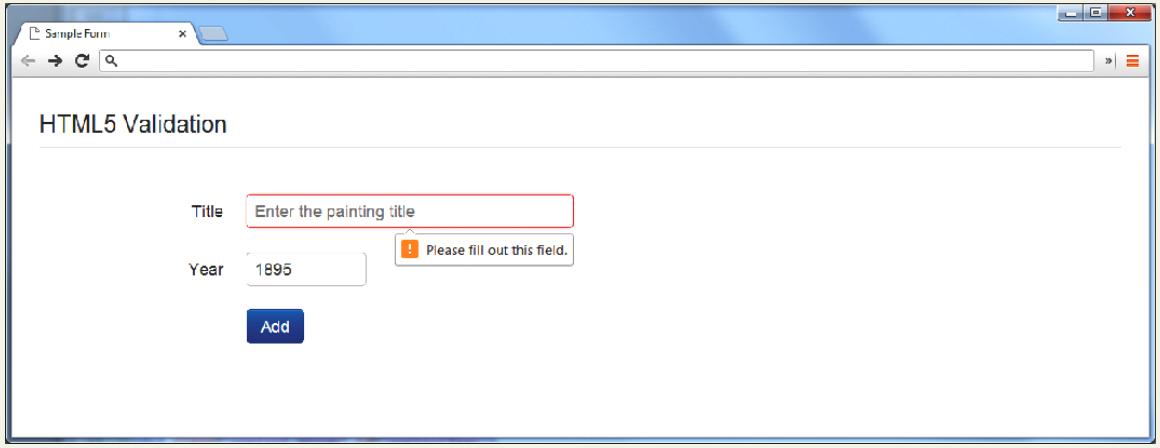
- Client-side using HTML5
- Client-Side using JavaScript
- **Server-Side using PHP**

While both client and server side validation is ideal, you must know that client-side scripts are not guaranteed to be executed. Therefore **you must always perform server-side validation**.

HTML5 validation

Client-Side

The ***required*** attribute can be added to an input element, and browsers that support it will perform their own validation and message.



To disable HTML form validation

<form id="sampleForm" method=". . ." action=". . ." *novalidate*>

JavaScript validation

Client-Side

Consider that we want to validate on a form submit.

```
function init() {
    const sampleForm = document.getElementById('sampleForm');
    sampleForm.onsubmit = validateForm;

}
 // call the init function once all the html has been loaded
window.onload = init;
```

JavaScript validation

Client-Side

For instance, to check if the value in the form's password input element is between 9 and 17 characters (and the first is a letter), the JavaScript would be:

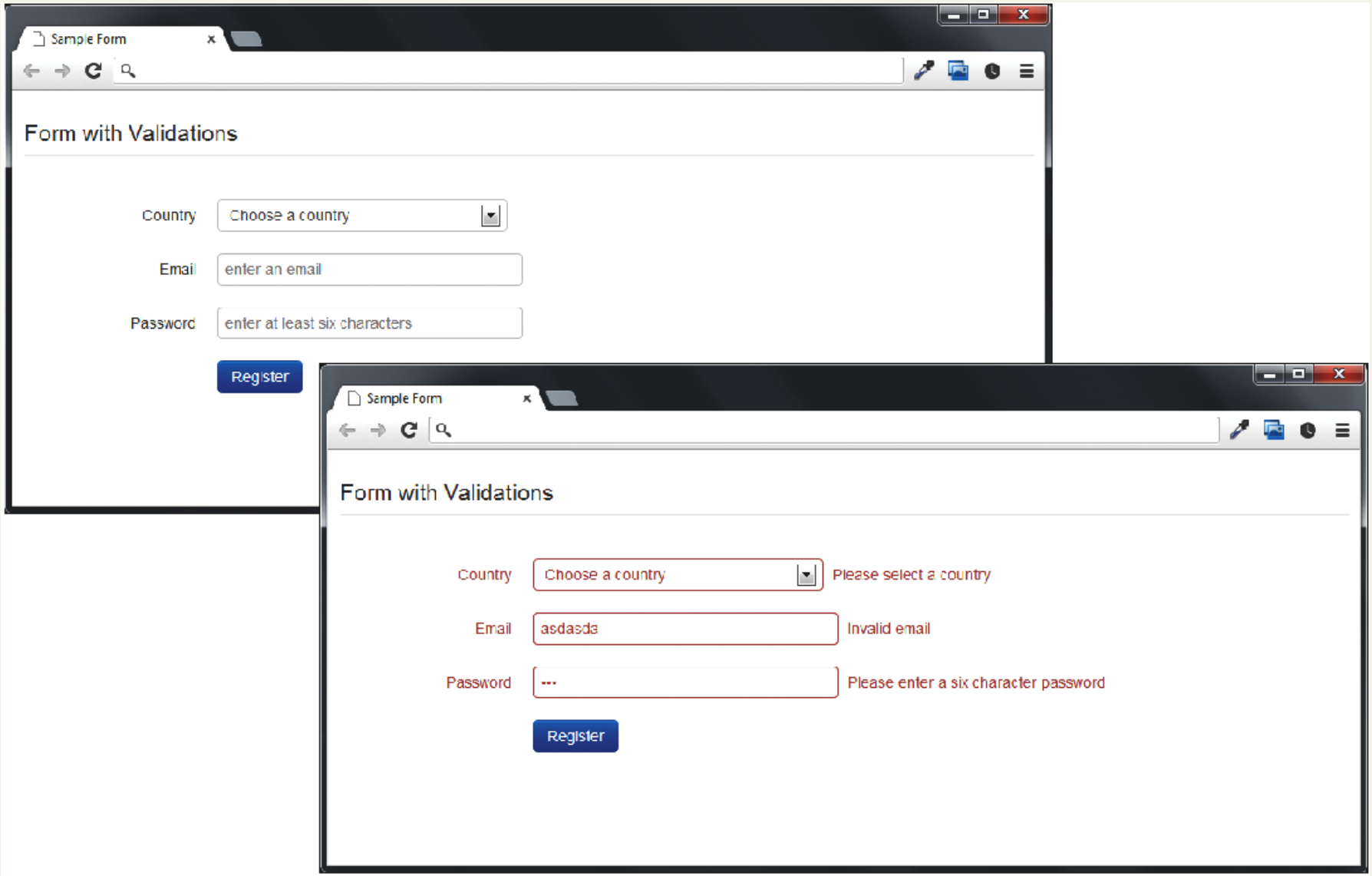
```
const passReg = /^[a-zA-Z]\w{8,16}$/;
if (! passReg.test(password.value)) {
    // provide some type of error message
}
```

What do we want to do when the JavaScript finds a validation error?

- Highlight errors by **adding CSS classes** to the input elements causing the error

JavaScript validation

Client-Side



A simple example

```
<!DOCTYPE html>
<head>
  <meta charset="utf-8">
  <title>Esempio validazione</title>
  <script src="./valid.js"></script>
  <link rel="stylesheet" href="./mystyle.css">
</head>
<body>
<form id="buy" action="order.php" method="get">
  <!-- First input is validated using HTML5 -->
  <label for="quante">How many eggs?</label>
  <input type="number" min="6" max="60"
   step="6" id="quante" name="ne" required>
  <br>
```

A simple example

```
  <!-- Second input is validated using JS -->
  <label for="dove1">Shipping address (street)</label>
  <input type="text" id="dove1" name="sa"
       placeholder="via Diotisalvi" required>
  <br>
  <!-- Third input is validated using HTML5 -->
  <label for="dove2">Postal code</label>
  <input type="text" id="dove2" name="pc"
       pattern="^[0-9]{5}$" placeholder="56122" required>
  <br>
  <!-- The fourth field is not required and not validated -->
  <label for="note">Additional info for the delivery:</label>
  <input type="text" id="note"><br>
  <button type="submit">Submit</button>
</form>
</body>
```

A simple example

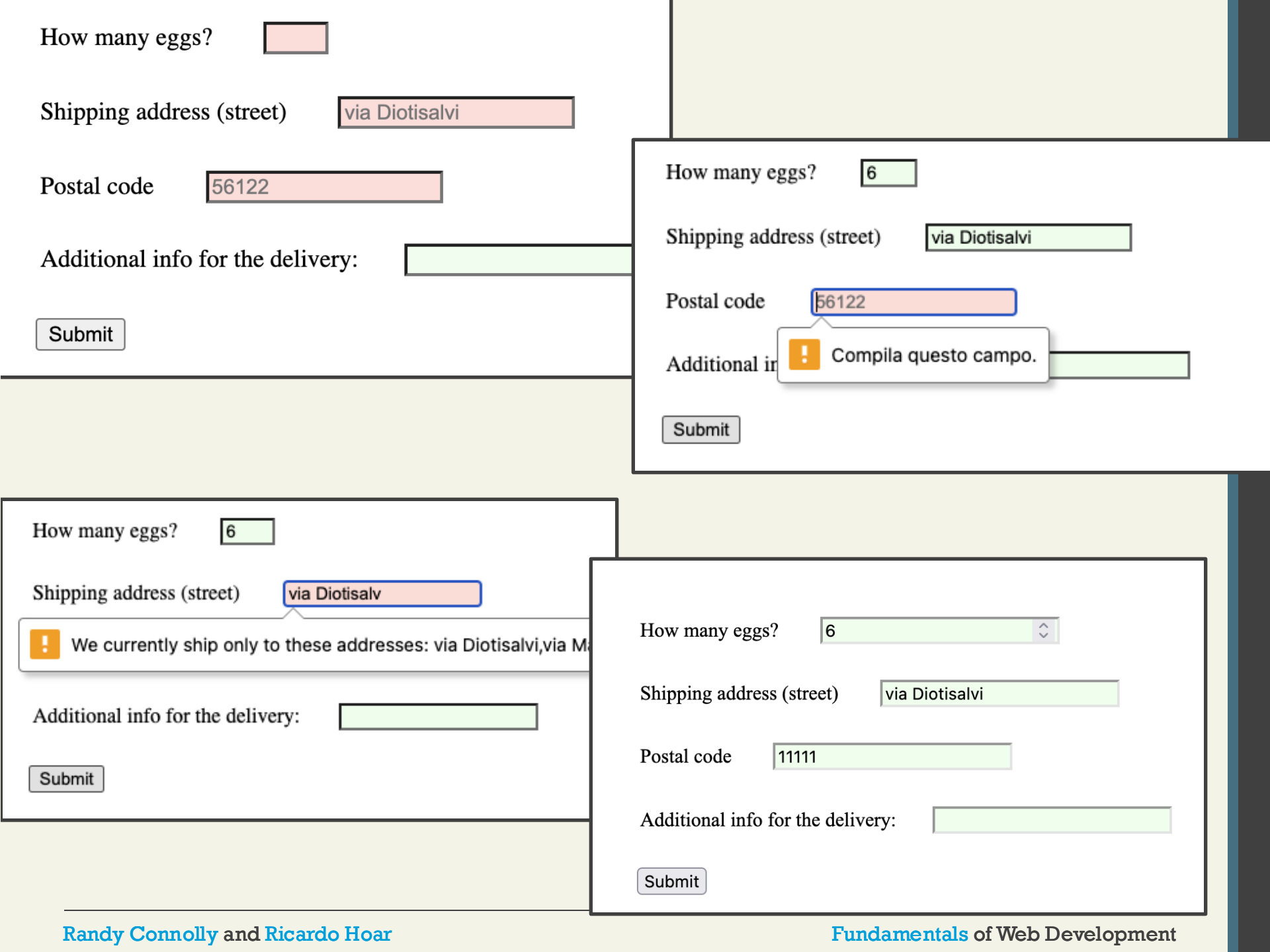
```
* {margin: 1em;}
/*
:invalid is a pseudo class defined by HTML5
it is automatically applied when the constraints specified
by means of HTML5 attributes are not met
*/
input:invalid {background-color: rgba(255, 0, 0, 0.167);}
/*
:valid and :required are two pseudoclasses, same as before */
input:valid{background-color: rgba(152, 251, 152, 0.171);}
```

A simple example

```
// valid.js
function validate(e) {
  // Only the second field is validated by means of JS code
  const sa = document.getElementById("dove1");
  const v = sa.value;
  const addr = ["via Diotisalvi", "via Mazzini",
                "Corso Italia"];
  if (!addr.includes(v)) {
    let msg =
      "We currently ship only to these addresses: " + addr;
    sa.setCustomValidity(msg);
  } else {
    sa.setCustomValidity("");
  }
}

function init() {
  const f = document.getElementById("dove1");
  f.addEventListener("input", validate);
}

window.onload = init;
```


How many eggs?
Shipping address (street)
via Diotisalvi
Postal code
56122
Additional info for the delivery:
Submit
How many eggs?
6
Shipping address (street)
via Diotisalvi
Postal code
56122
Compila questo campo.
Submit
How many eggs?
6
Shipping address (street)
via Diotisalv
We currently ship only to these addresses: via Diotisalvi,via M
Additional info for the delivery:
Submit
How many eggs?
6
Shipping address (street)
via Diotisalvi
Postal code
11111
Additional info for the delivery:
Submit

A simple example

```php
<?php
if(empty($_GET["ne"]) ||
   empty($_GET["sa"]) ||
   empty($_GET["pc"])) {
  echo "One of the required values is missing";
  die();
}
$ne = $_GET["ne"];
$sa = $_GET["sa"];
$pc = $_GET["pc"];

if(!is_numeric($ne) || $ne % 6 != 0 ||
    $ne < 6 || $ne > 60) {
  echo "The number of eggs must be 6, 12, 18, ..., 60";
  die();
}
```

A simple example

```
$addr = ["via Diotisalvi", "via Mazzini", "Corso Italia"];
if(!in_array($sa, $addr)) {
  echo "We currently ship only to these addresses: ";
  foreach($addr as $a)
    echo $a . " ";
  die();
}

if(!is_numeric($pc) || preg_match('/^[0-9]{5}$/', $pc)!=1) {
  echo "The postal code is not correct";
  die();
}

echo "Parameters are OK!";
?>
```

